

LIBRARY AUTOMATION SYSTEM

A Project Report

Submitted in Partial Fulfilment

of the Degree of

Bachelor of Computer Applications

Supervisor's Name:

Prof. Kavita Mittal

Submitted By:

Student Name: Khushboo Masson

Enrolment No.: 120925198

Semester: II



JAGANNATH UNIVERSITY
BAHADURGARH (DELHI NCR)
(2025-28)

PROJECT CERTIFICATE

This is to certify that the project report entitled “LIBRARY AUTOMATION SYSTEM” submitted to Jagannath University, Delhi NCR, Bahadurgarh in partial fulfilment of the requirement for the award of the degree of BACHELOR OF COMPUTER APPLICATIONS (BCA), is an original work carried out by Ms. Khushboo Masson, Enrolment No. (University): 120925198, under the guidance of Prof. Kavita Mittal.

The matter embodied in this project is a genuine work done by the student and has not been submitted, whether to this University or to any other University/Institute, for the fulfilment of the requirement of any course of study.

Name of the Guide: Prof. Kavita
Mittal

Name of the Student: Khushboo
Masson

Signature of the Guide:

Signature of the Student:

Date: _____

Enrolment No.: 120925198

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, Prof. Kavita Mittal, for her valuable guidance, continuous encouragement, and constructive suggestions provided throughout the course of this project, “Library Automation System”.

I am also thankful to the Department of Computer Applications, Jagannath University, Bahadurgarh (Delhi NCR), for providing the necessary infrastructure, resources, and a conducive environment to carry out this project successfully.

I extend my heartfelt thanks to my family and friends for their constant support and motivation during the completion of this project. Finally, I am grateful to all the faculty members who directly or indirectly helped me in completing this project report.

Student Name: Khushboo Masson

Enrolment No.: 120925198

INDEX

S.No.	Topic	Page No.
1	Introduction	5
2	Objectives	7
3	Tools / Environment	9
4	Flow Chart	10
5	Program Code	13
6	Validation	19
7	Input and Output Screens	21
8	Limitations of the Project	28
9	Future Application of the Project	30
10	Bibliography	31

INTRODUCTION

The “Library Automation System” is a computerised application designed to manage the day-to-day operations of a library efficiently, replacing the traditional manual, register-based method of record keeping. In a conventional library, tasks such as cataloguing books, issuing and returning books, tracking overdue fines, and maintaining borrower records are performed manually, which is time-consuming, error-prone, and difficult to scale as the collection of books and the number of members grows.

This project automates the core functions of a small library, including maintaining a book catalogue, borrowing and returning books, automatic escalating fine calculation for late returns, and account management for both students and teachers. The system is implemented in Python as a menu-driven, terminal-based application that stores all of its data — books, student and teacher accounts, loan records, and fines — in a structured JSON file (`library_data.json`) using Python's built-in `json` module. This removes the need to install or configure a separate database server, while still allowing all data to persist between sessions. The system also transparently migrates data saved by older versions of the program — for example, assigning a valid roll number to a student record that predates that field — so existing data is never lost when the program is upgraded.

The system supports three roles. A Student logs in with a roll number (which must begin with 120925) and a password to browse the catalogue, borrow up to three books at a time, return them, and track their own active loans, overdue books, and outstanding fines. A Teacher logs in with a staff ID in the format `TCH-#####` and a password, and has the same borrowing menu as a student but without the three-book limit. An Admin logs in with a password to view the complete catalogue together with the name and ID of every current borrower, review every loan and fine on record, look up any individual student or teacher by their ID alone, and register new student or teacher accounts directly from the

terminal. Fines are no longer a flat rate: the system automatically recalculates every open loan's fine each time the program starts or a menu is used, applying ₹50 for the first week a book is overdue and an additional ₹15 for every further week, and finalising that amount the moment the book is returned.

The Library Automation System is intended as a foundation project demonstrating core programming concepts — data persistence, data migration, role-based access, regular-expression input validation, date-based business logic, and a structured menu-driven interface — and can be extended in the future with a graphical or web-based interface, a relational database backend, and additional automation features, making it a scalable starting point for a fuller Integrated Library Management System (ILMS).

OBJECTIVES

The primary objectives of the Library Automation System project are as follows:

- To design a centralised, file-based data store (JSON) for accurately storing and managing book, student, teacher, loan, and fine records, including automatic migration of records saved by earlier versions of the program.
- To automate the process of issuing (borrowing) and returning books, reducing manual paperwork and register entries.
- To provide Students and Teachers with direct, self-service access to the book catalogue, their own borrowing history, active loans, and outstanding fines, secured by an ID-and-password login.
- To identify Students by a validated roll number (beginning 120925) and Teachers by a validated staff ID (format TCH-#####), rejecting any ID that does not match the required pattern when an account is created.
- To limit Students to a maximum of three books borrowed at any one time, while allowing Teachers to borrow without that restriction.
- To maintain an up-to-date record of the availability status of each book, including the name and ID of every person currently holding a copy.
- To calculate fines automatically and progressively for overdue books — ₹50 for the first week overdue, plus ₹15 for every additional week — and to keep every open loan's fine current every time the program is used, without waiting for the book to be returned.
- To give library administrators a consolidated view of the entire catalogue (with current borrowers), every loan, and every overdue loan across all students and teachers.

- To allow administrators to register new student and teacher accounts, and to look up any student or teacher's outstanding fines using only their ID.
- To provide a simple, role-based, menu-driven interface (Student / Teacher / Admin) that reduces the learning curve for new users.
- To ensure data is saved immediately after every change, so that no information is lost between program runs.
- To create a system that can be extended in future with additional features such as a graphical interface, a proper database backend, and search/notification facilities.

TOOLS / ENVIRONMENT

3.1 Software Requirements

- Programming Language: Python 3, implemented as a menu-driven command-line (terminal-based) application.
- Data Storage: A structured JSON file (library_data.json), read and written using Python's built-in json module — no external database server is required.
- Standard Library Modules: json and os for data storage, re for validating roll numbers and teacher IDs, and datetime (date, timedelta) for due dates and fine calculation.
- Operating System: Windows / Linux — the application depends only on the standard Python interpreter, so it runs unmodified on Windows PowerShell or a Linux terminal.
- Execution Environment: A terminal / shell, run using the command: python main.py (Windows PowerShell) or python3 main.py (Linux bash).
- Editor: VS Code / nano.
- Documentation Tool: Microsoft Word / LibreOffice Writer.
- Diagramming Tool: Graphviz, used to generate the process flowcharts included in this report.

3.2 Hardware Requirements

- Processor: Intel Core i3 or above.
- RAM: 4 GB or above.
- Hard Disk: 20 GB free space recommended.
- Keyboard and Monitor — no special graphics hardware is required, since the application runs entirely in the terminal.

FLOW CHART

The flowcharts below represent the process flow of the Library Automation System. Fig. 4.1 shows the overall system flow from program start to exit, including the Student / Teacher / Admin login and the automatic fine refresh that runs every time a menu loads. Fig. 4.2 and Fig. 4.3 expand the Student/Teacher Account Menu and the Admin Menu respectively, showing the decision logic — including the borrowing limit and the escalating fine calculation — behind each individual operation.

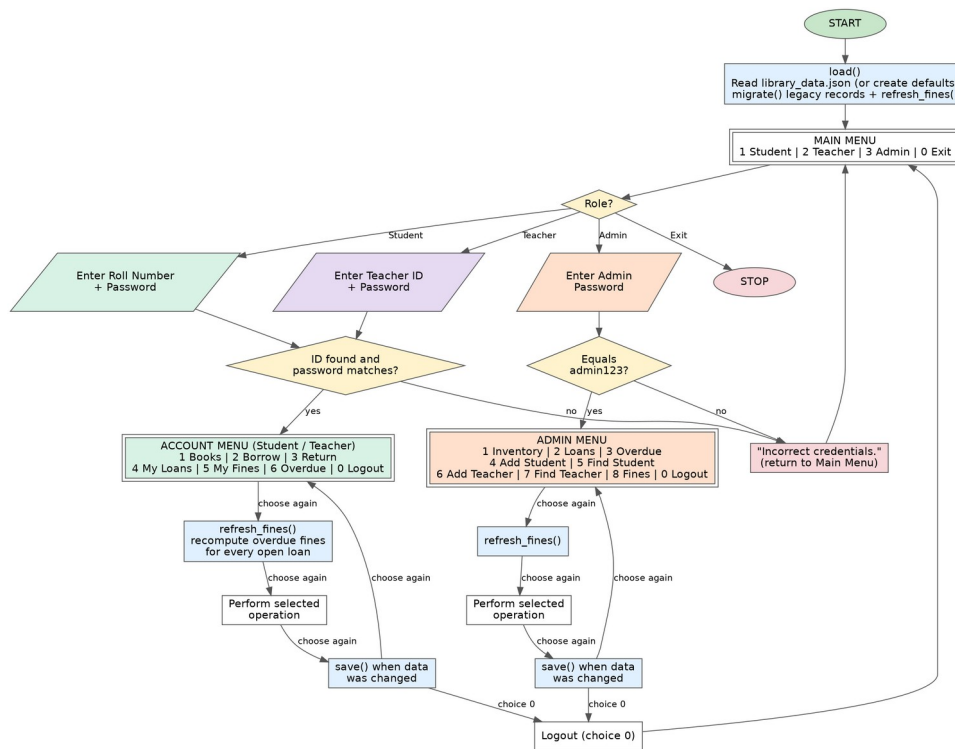


Fig 4.1: Overall Flow of the Library Automation System

The diagram below expands the Student/Teacher Account Menu, showing the borrowing limit check, book-availability validation, and the escalating fine calculation applied on return.

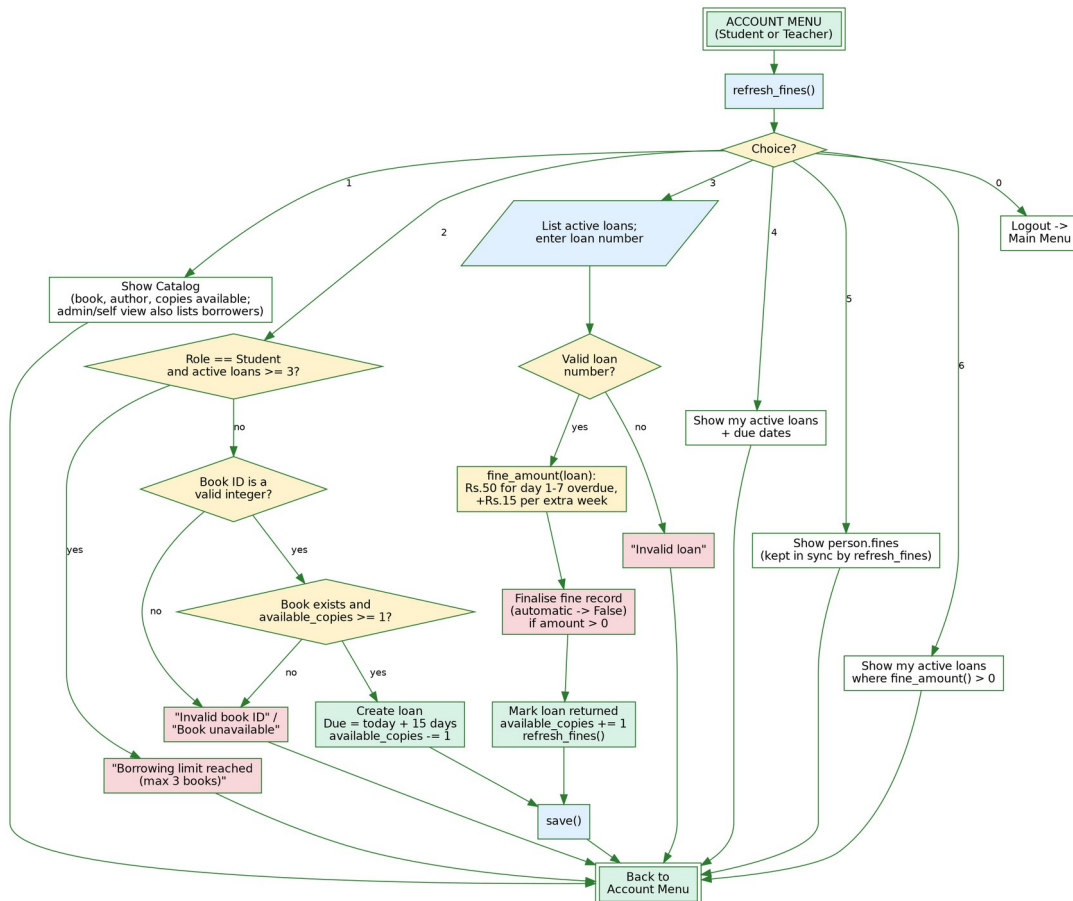


Fig 4.2: Student / Teacher Account Menu — Detailed Operation Flow

The diagram below expands the Admin Menu, showing the inventory, loan-record, account-creation, and lookup operations available to the administrator.

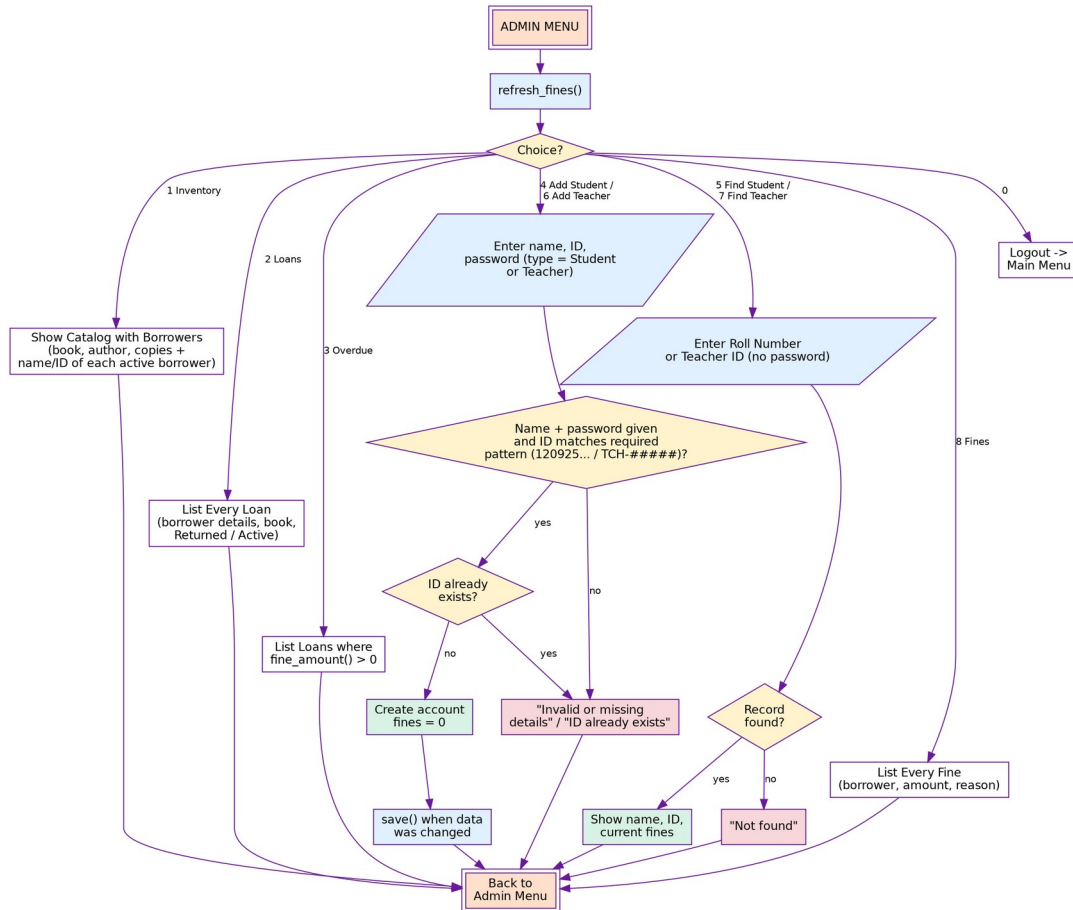


Fig 4.3: Admin Menu — Detailed Operation Flow

PROGRAM CODE

The Library Automation System is implemented in Python as a single-file, menu-driven command-line application (main.py). It uses Python's built-in json module for data storage and the re module to validate roll numbers and teacher IDs, so no separate database server or third-party library needs to be installed. The complete source code is organised into functions for setup and persistence, legacy-data migration, automatic fine calculation, data loading and lookup, catalogue display, account creation, the combined Student/Teacher account menu, the Admin menu, and a main() driver function. The complete code is reproduced below.

5.1 Setup, Constants, and Data Persistence

These lines define the data file name, the regular expressions used to validate a Student roll number and a Teacher ID, the default dataset for a first run, and the function that writes the current data back to library_data.json.

```
import json, os, re
from datetime import date, timedelta

DATA_FILE = "library_data.json"
ROLL = re.compile(r"^120925\d+$")
TEACHER = re.compile(r"^TCH-\d{5}$", re.I)

def initial_data():
    return {"books": [{"id": 1, "title": "Python Basics", "author": "A. Smith",
"genre": "Programming", "total_copies": 3, "available_copies": 3}, {"id": 2,
"title": "Data Structures", "author": "M. Khan", "genre": "Computer Science",
"total_copies": 2, "available_copies": 2}, {"id": 3, "title": "The Great
Gatsby", "author": "F. Scott Fitzgerald", "genre": "Classic", "total_copies":
2, "available_copies": 2}], "students": [], "teachers": [], "loans": [],
" fines": []}

def save(data):
    with open(DATA_FILE, "w", encoding="utf-8") as f: json.dump(data, f,
indent=2)
```

5.2 Legacy Data Migration

legacy_roll() invents the next free roll number for an old-style student record. migrate() upgrades data saved by earlier versions of the program — assigning roll numbers, defaulting missing fines to zero, and backfilling

borrower_type/borrower_id/borrower_name on older loan and fine records — so nothing is lost when the program is updated.

```
def legacy_roll(students):
    n, used = 1, {x.get("roll_number") for x in students}
    while f"120925{n:03d}" in used: n += 1
    return f"120925{n:03d}"

def migrate(data):
    changed = False
    data.setdefault("students", []); data.setdefault("teachers", []);
    data.setdefault("loans", []); data.setdefault("fines", [])
    for s in data["students"]:
        if "roll_number" not in s: s["roll_number"], changed =
legacy_roll(data["students"]), True
    s.setdefault("fines", 0)
    for t in data["teachers"]:
        t.setdefault("fines", 0)
        if t.get("teacher_id"): t["teacher_id"] = t["teacher_id"].upper()
    names = {s["name"].lower(): s for s in data["students"]}
    for num, loan in enumerate(data["loans"], 1):
        if "id" not in loan: loan["id"], changed = num, True
        if "borrower_type" not in loan:
            s = names.get(loan.get("student_name", "").lower())
            loan.update({"borrower_type": "student",
"borrower_id": s["roll_number"] if s else "UNKNOWN",
"borrower_name": loan.get("student_name", "Unknown")}); changed = True
        for fine in data["fines"]:
            if "borrower_type" not in fine:
                s = names.get(fine.get("student_name", "").lower())
                fine.update({"borrower_type": "student",
"borrower_id": s["roll_number"] if s else "UNKNOWN",
"borrower_name": fine.get("student_name", "Unknown")}); changed = True
    return changed
```

5.3 Automatic Fine Calculation

fine_amount() computes what a single loan currently owes: ₹50 once it is 1–7 days overdue, plus a further ₹15 for every additional full week. refresh_fines() re-runs this for every open loan each time it is called, creates or updates an “automatic” fine record for each one, removes automatic fines for loans that are no longer overdue, and keeps every student's and teacher's total fines field in sync.

```
def fine_amount(loan, today=None):
    if loan.get("returned"): return 0
    overdue = ((today or date.today()) -
date.fromisoformat(loan["due_date"])).days
    return 0 if overdue <= 0 else 50 + ((overdue - 1) // 7) * 15

def refresh_fines(data):
    changed, today, active_ids = False, date.today(), set()
    for loan in data["loans"]:
        amount = fine_amount(loan, today)
        if not amount: continue
        active_ids.add(loan["id"])
```

```

        old = next((x for x in data["fines"] if x.get("automatic") and
x.get("loan_id") == loan["id"]), None)
        if old is None:
            data["fines"].append({"borrower_type":loan["borrower_type"],
"borrower_id":loan["borrower_id"], "borrower_name":loan["borrower_name"],
"loan_id":loan["id"], "automatic":True, "amount":amount, "reason":f"Overdue:
{loan['book_title']}"}); changed = True
            elif old["amount"] != amount: old["amount"], changed = amount, True
            before = len(data["fines"])
            data["fines"][:] = [x for x in data["fines"] if not x.get("automatic") or
x.get("loan_id") in active_ids]
            changed |= len(data["fines"]) != before
            for typ, key in (("student", "roll_number"), ("teacher", "teacher_id")):
                for p in data[typ + "s"]:
                    total = sum(x["amount"] for x in data["fines"] if
x.get("borrower_type") == typ and x.get("borrower_id", "").upper() ==
p[key].upper())
                    if p.get("fines") != total: p["fines"], changed = total, True
            if changed: save(data)

```

5.4 Loading Data and Lookup Helpers

`load()` reads `library_data.json` (or creates the defaults), runs the migration and an initial fine refresh, and saves the result if anything changed. `find()` looks up a student or teacher by roll number / teacher ID, and `borrower_details()` formats a loan or fine's borrower for display.

```

def load():
    if os.path.exists(DATA_FILE):
        with open(DATA_FILE, encoding="utf-8") as f: data = json.load(f)
    else: data = initial_data()
    changed = migrate(data); refresh_fines(data)
    if changed: save(data)
    return data

def find(data, typ, ident):
    key = "roll_number" if typ == "student" else "teacher_id"
    return next((x for x in data[typ + "s"] if x.get(key, "").upper() ==
ident.upper()), None)

def borrower_details(loan):
    """Return the appropriate borrower name and identifier for catalog
displays."""
    if loan["borrower_type"] == "student":
        return f"Student name: {loan['borrower_name']} | Enrollment number:
{loan['borrower_id']}"
    return f"Teacher name: {loan['borrower_name']} | Teacher ID:
{loan['borrower_id']}"

```

5.5 Catalogue Display

`show_books()` prints every book with its author and available copies. When called for a logged-in student or teacher it also prints their own name and ID; when called by the admin with `show_borrowers=True` it lists the name and ID of every current borrower under each book.

```

def show_books(data, person=None, typ=None, show_borrowers=False):
    print("\nCatalog")
    if person and typ == "student":
        print(f"Student name: {person['name']} | Enrollment number: {person['roll_number']}")
    elif person and typ == "teacher":
        print(f"Teacher name: {person['name']} | Teacher ID: {person['teacher_id']}")

    for b in data["books"]:
        print(f"{b['id']}. {b['title']} | {b['author']} | Available: {b['available_copies']}")
        if show_borrowers:
            borrowers = [x for x in data["loans"] if x["book_id"] == b["id"] and not x["returned"]]
            for loan in borrowers:
                print(f"    Borrowed by: {borrower_details(loan)}")

```

5.6 Account Creation

`add_account()` is shared by both “Add a student” and “Add a teacher” in the Admin menu. It rejects an empty name/password, an ID that does not match the required pattern, or an ID that is already registered, before creating the new account.

```

def add_account(data, typ):
    name = input(f"Enter {typ} name: ").strip()
    label, pattern = ("roll number (must begin 120925)", ROLL) if typ == "student" else ("teacher ID (e.g. TCH-00125)", TEACHER)
    ident = input(f"Enter {label}: ").strip().upper() if typ == "teacher" else input(f"Enter {label}: ").strip()
    password = input(f"Enter {typ} password: ").strip()
    if not name or not password or not pattern.fullmatch(ident): print("Invalid or missing details."); return
    if find(data, typ, ident): print("That ID already exists."); return
    data[typ + "s"].append({"name":name, "password":password, "fines":0, "roll_number":ident} if typ == "student" else {"name":name, "password":password, "fines":0, "teacher_id":ident})
    save(data); print(f"{typ.title()} account created: {name} ({ident})")

```

5.7 Student / Teacher Account Menu

`account_menu()` drives every Student and Teacher operation: refreshing fines on entry, listing the catalogue, borrowing (with the three-book limit for students and the book-availability check), returning (with the escalating fine finalised at the moment of return), and viewing loans, fines, and overdue books.

```

def account_menu(data, person, typ):
    key, ident = ("roll_number", person["roll_number"]) if typ == "student" else ("teacher_id", person["teacher_id"])
    while True:
        refresh_fines(data)
        if typ == "student":
            print(f"\nStudent account: {person['name']} | Enrollment number: {ident}")
        else:

```

```

        print(f"\nTeacher account: {person['name']} | Teacher ID: {ident}")
        print(f"\n{typ.title()} menu: 1 Books 2 Borrow 3 Return 4 My loans
5 My fines 6 Overdue 0 Logout")
        choice = input("Choice: ").strip()
        active = [x for x in data["loans"] if x["borrower_type"] == typ and
x["borrower_id"].upper() == ident.upper() and not x["returned"]]
        if choice == "1": show_books(data, person, typ)
        elif choice == "2":
            if typ == "student" and len(active) >= 3:
                print("Borrowing limit reached: students can borrow a maximum
of 3 books at a time.")
                continue
            show_books(data, person, typ)
            try: book_id = int(input("Book ID: "))
            except ValueError: print("Invalid book ID."); continue
            book = next((b for b in data["books"] if b["id"] == book_id), None)
            if not book or book["available_copies"] < 1: print("Book
unavailable."); continue
            due = date.today() + timedelta(days=15)
            data["loans"].append({"id":max((x["id"] for x in data["loans"]),
default=0)+1, "borrower_type":typ, "borrower_id":ident,
"borrower_name":person["name"], "book_id":book_id, "book_title":book["title"],
"borrowed_date":date.today().isoformat(), "due_date":due.isoformat(),
"returned":False})
            book["available_copies"] -= 1; save(data); print(f"Borrowed. Return
by {due}.")
            elif choice == "3":
                for n, loan in enumerate(active, 1): print(f"{n}.
{loan['book_title']} (due {loan['due_date']})")
                try: loan = active[int(input("Loan number: ")) - 1]
                except (ValueError, IndexError): print("Invalid loan."); continue
                amount = fine_amount(loan)
                if amount:
                    current = next((x for x in data["fines"] if x.get("automatic")
and x.get("loan_id") == loan["id"]), None)
                    if current: current.update({"automatic":False, "amount":amount,
"reason":f"Late return: {loan['book_title']}"})
                    else: data["fines"].append({"borrower_type":typ,
"borrower_id":ident, "borrower_name":person["name"], "loan_id":loan["id"],
"automatic":False, "amount":amount, "reason":f"Late return:
{loan['book_title']}"})
                    loan["returned"], loan["returned_date"] = True,
date.today().isoformat()
                    next(b for b in data["books"] if b["id"] == loan["book_id"])
                    ["available_copies"] += 1
                    refresh_fines(data); save(data); print(f"Returned. Fine: Rs.
{amount}")
            elif choice == "4":
                for x in active: print(f"{x['book_title']} | Due: {x['due_date']}")
                if not active: print("No active loans.")
            elif choice == "5": print(f"Outstanding fines: Rs. {person['fines']}")
            elif choice == "6":
                overdue = [x for x in active if fine_amount(x)]
                for x in overdue: print(f"{x['book_title']}: Rs. {fine_amount(x)}")
                if not overdue: print("No overdue books.")
            elif choice == "0": return
            else: print("Invalid choice.")

```

5.8 Admin Menu

admin_menu() drives every administrator operation: the annotated inventory, every loan record, every overdue loan, creating a student or teacher account, looking up a student or teacher by ID alone, and viewing every recorded fine.

```

def admin_menu(data):
    while True:
        refresh_fines(data)
        print("\nAdmin: 1 Inventory  2 Loans  3 Overdue  4 Add student  5 Find
student by roll no  6 Add teacher  7 Find teacher  8 Fines  0 Logout")
        choice = input("Choice: ").strip()
        if choice == "1": show_books(data, show_borrowers=True)
        elif choice == "2":
            for x in data["loans"]: print(f"{borrower_details(x)} |
{x['book_title']} | {'Returned' if x['returned'] else 'Active'}")
        elif choice == "3":
            for x in data["loans"]:
                if fine_amount(x): print(f"{borrower_details(x)} |
{x['book_title']} | Rs. {fine_amount(x)}")
        elif choice == "4": add_account(data, "student")
        elif choice == "5":
            s = find(data, "student", input("Student roll number: ").strip())
            print(f"{s['name']} | {s['roll_number']} | Fines: Rs. {s['fines']}")
            if s else "Student not found."
        elif choice == "6": add_account(data, "teacher")
        elif choice == "7":
            t = find(data, "teacher", input("Teacher ID: ").strip())
            print(f"{t['name']} | {t['teacher_id']} | Fines: Rs. {t['fines']}")
            if t else "Teacher not found."
        elif choice == "8":
            for x in data["fines"]: print(f"{borrower_details(x)} | Rs.
{x['amount']} | {x['reason']}")
        elif choice == "0": return
        else: print("Invalid choice.")

```

5.9 Main Driver Function

main() loads the data file and displays the top-level menu that routes the user into the Student, Teacher, or Admin flow based on role and credentials.

```

def main():
    data = load(); print("Welcome to the Library Management System")
    while True:
        role = input("\n1 Student  2 Teacher  3 Admin  0 Exit\nChoice:
").strip()
        if role == "0": return
        if role == "3":
            if input("Admin password: ").strip() == "admin123":
                admin_menu(data)
            else: print("Incorrect credentials.")
            continue
        typ = "student" if role == "1" else "teacher" if role == "2" else None
        if not typ: print("Invalid choice."); continue
        ident = input("Roll number: " if typ == "student" else "Teacher ID:
").strip()
        person = find(data, typ, ident)
        if person and person["password"] == input("Password: ").strip():
            account_menu(data, person, typ)
        else: print("Incorrect credentials.")

if __name__ == "__main__": main()

```

The application is executed from the terminal as follows:

```
khushboo@ubuntu:~/library_automation_system$ python3 main.py
```

VALIDATION

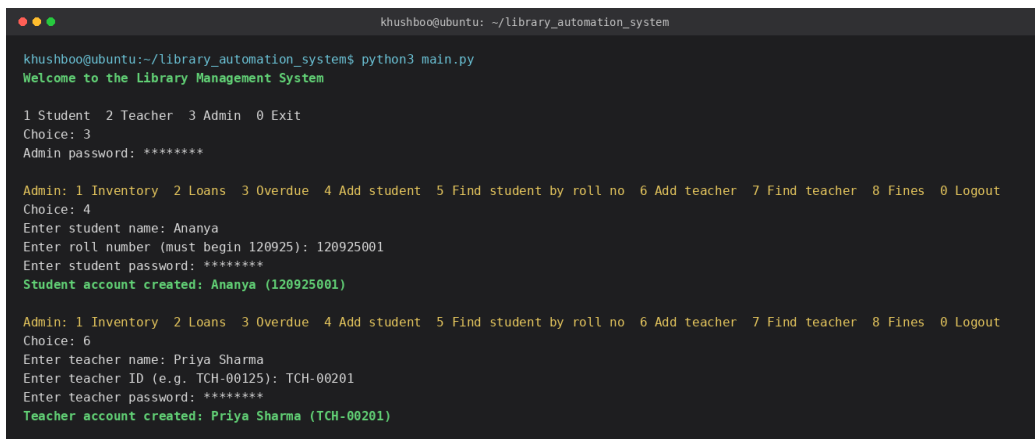
The following validation checks are built into the current implementation to keep the data consistent:

- Student and Teacher login requires both a matching ID (roll number or teacher ID) and password; Admin login requires the correct password. Incorrect credentials show “Incorrect credentials.” and return the user to the main menu without granting access.
- A new student's roll number must fully match `^120925\d+$` and a new teacher's ID must fully match `^TCH-\d{5}$` (case-insensitive, stored uppercase); an account is rejected with “Invalid or missing details.” if the name, password, or ID pattern is not satisfied.
- Creating a student or teacher account is rejected with “That ID already exists.” if the roll number or teacher ID is already registered.
- Students are limited to three active (not-yet-retuned) loans at a time; a fourth borrow attempt is rejected with “Borrowing limit reached” before a Book ID is even requested. Teachers are not subject to this limit.
- A Book ID entered when borrowing is read with `int()` inside a try/except, so non-numeric input is rejected as “Invalid book ID.” instead of crashing the program; a valid but unknown ID, or a book with no available copies, is rejected as “Book unavailable.”
- A loan number entered when returning a book is validated against the student/teacher's own active loans (out-of-range or non-numeric input shows “Invalid loan.”), so a book cannot be returned twice or on behalf of someone else.
- Fines are calculated purely from the due date and today's date (`fine_amount()`); a loan that is not yet overdue, or has already been returned, always evaluates to zero, so on-time returns are never fined.

- The escalating fine — ₹50 for the first overdue week, +₹15 per additional week — is recalculated by `refresh_fines()` every time the program loads or a Student/Teacher/Admin menu is used, so a displayed fine is always current even if the book has not yet been returned.
- When a book is returned, any automatic fine tied to that loan is finalised (automatic set to False) at the exact amount owed at that moment, so the fine no longer changes after the book is back on the shelf.
- Admin lookup of a student or teacher (“Find a student” / “Find a teacher”) only requires the ID, and reports “Not found.” for an ID that is not registered.
- Every state-changing action (borrowing, returning, creating an account) calls `save()` so changes are written to `library_data.json` immediately and are not lost if the program is closed.
- If `library_data.json` is missing, or contains records saved by an older version of the program, `load()` creates sensible defaults or migrates the old records automatically rather than crashing on startup.

INPUT AND OUTPUT SCREENS

The screenshots below were captured from a live run of the application (python3 main.py) in a Linux terminal, and illustrate the menu, input prompts, and output/result displays of every Student, Teacher, and Admin operation in the Library Automation System, in the order they were performed during testing.



```
khushboo@ubuntu: ~/library_automation_system

khushboo@ubuntu:~/library_automation_system$ python3 main.py
Welcome to the Library Management System

1 Student  2 Teacher  3 Admin  0 Exit
Choice: 3
Admin password: *****

Admin: 1 Inventory  2 Loans  3 Overdue  4 Add student  5 Find student by roll no  6 Add teacher  7 Find teacher  8 Fines  0 Logout
Choice: 4
Enter student name: Ananya
Enter roll number (must begin 120925): 120925001
Enter student password: *****
Student account created: Ananya (120925001)

Admin: 1 Inventory  2 Loans  3 Overdue  4 Add student  5 Find student by roll no  6 Add teacher  7 Find teacher  8 Fines  0 Logout
Choice: 6
Enter teacher name: Priya Sharma
Enter teacher ID (e.g. TCH-00125): TCH-00201
Enter teacher password: *****
Teacher account created: Priya Sharma (TCH-00201)
```

Fig 7.1: Welcome Screen, Main Menu, and Admin Creating a Student and a Teacher Account

```
khushboo@ubuntu: ~/library_automation_system

1 Student  2 Teacher  3 Admin  0 Exit
Choice: 1
Roll number: 120925001
Password: *****

Student account: Ananya | Enrollment number: 120925001

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 1

Catalog
Student name: Ananya | Enrollment number: 120925001
1. Python Basics | A. Smith | Available: 3
2. Data Structures | M. Khan | Available: 2
3. The Great Gatsby | F. Scott Fitzgerald | Available: 2

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 2
Book ID: 2
Borrowed. Return by 2026-09-19.

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 4
Data Structures | Due: 2026-09-19
```

Fig 7.2: Student Login (Roll Number + Password), Catalogue, Borrow a Book, and My Loans

```
khushboo@ubuntu: ~/library_automation_system

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 2
Book ID: 3
Borrowed. Return by 2026-09-19.

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 2
Book ID: 1
Borrowed. Return by 2026-09-19.

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 2
Borrowing limit reached: students can borrow a maximum of 3 books at a time.
```

Fig 7.3: Borrowing a Second and Third Book, then the Borrowing Limit Being Enforced

```
khushboo@ubuntu: ~/library_automation_system

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 6
Data Structures: Rs. 65

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 3
1. Data Structures (due 2026-08-25)
2. The Great Gatsby (due 2026-09-19)
3. Python Basics (due 2026-09-19)
Loan number: 1
Returned. Fine: Rs. 65

Student menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 5
Outstanding fines: Rs. 65
```

Fig 7.4: Overdue View, Returning a Late Book with the Escalating Fine, and My Fines

```
khushboo@ubuntu: ~/library_automation_system

1 Student  2 Teacher  3 Admin  0 Exit
Choice: 2
Teacher ID: TCH-00201
Password: *****

Teacher account: Priya Sharma | Teacher ID: TCH-00201

Teacher menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 1

Catalog
Teacher name: Priya Sharma | Teacher ID: TCH-00201
1. Python Basics | A. Smith | Available: 2
2. Data Structures | M. Khan | Available: 2
3. The Great Gatsby | F. Scott Fitzgerald | Available: 1

Teacher menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 2
Book ID: 1
Borrowed. Return by 2026-09-19.

Teacher menu: 1 Books  2 Borrow  3 Return  4 My loans  5 My fines  6 Overdue  0 Logout
Choice: 4
Python Basics | Due: 2026-09-19
```

Fig 7.5: Teacher Login (Teacher ID + Password), Catalogue, Borrow a Book, and My Loans

```
khushboo@ubuntu: ~/library_automation_system

Admin: 1 Inventory  2 Loans  3 Overdue  4 Add student  5 Find student by roll no  6 Add teacher  7 Find teacher  8 Fines  0 Logout
Choice: 1

Catalog
1. Python Basics | A. Smith | Available: 1
   Borrowed by: Student name: Ananya | Enrollment number: 120925001
   Borrowed by: Teacher name: Priya Sharma | Teacher ID: TCH-00201
2. Data Structures | M. Khan | Available: 2
3. The Great Gatsby | F. Scott Fitzgerald | Available: 1
   Borrowed by: Student name: Ananya | Enrollment number: 120925001

Admin: 1 Inventory  2 Loans  3 Overdue  4 Add student  5 Find student by roll no  6 Add teacher  7 Find teacher  8 Fines  0 Logout
Choice: 2
Student name: Ananya | Enrollment number: 120925001 | Data Structures | Returned
Student name: Ananya | Enrollment number: 120925001 | The Great Gatsby | Active
Student name: Ananya | Enrollment number: 120925001 | Python Basics | Active
Teacher name: Priya Sharma | Teacher ID: TCH-00201 | Python Basics | Active
```

Fig 7.6: Admin — Inventory with Current Borrowers, and All Loan Records

```
khushboo@ubuntu: ~/library_automation_system

Admin: 1 Inventory  2 Loans  3 Overdue  4 Add student  5 Find student by roll no  6 Add teacher  7 Find teacher  8 Fines  0 Logout
Choice: 5
Student roll number: 120925001
Ananya | 120925001 | Fines: Rs. 65

Admin: 1 Inventory  2 Loans  3 Overdue  4 Add student  5 Find student by roll no  6 Add teacher  7 Find teacher  8 Fines  0 Logout
Choice: 7
Teacher ID: TCH-00201
Priya Sharma | TCH-00201 | Fines: Rs. 0

Admin: 1 Inventory  2 Loans  3 Overdue  4 Add student  5 Find student by roll no  6 Add teacher  7 Find teacher  8 Fines  0 Logout
Choice: 8
Student name: Ananya | Enrollment number: 120925001 | Rs. 65 | Late return: Data Structures
```

Fig 7.7: Admin — Find a Student, Find a Teacher, and View All Fines

LIMITATIONS OF THE PROJECT

- The system currently supports a single library and a single terminal session at a time; it does not support simultaneous multi-user access over a network.
- Data is stored in a local JSON file rather than a relational database, which is simple and dependency-free but does not scale well to a very large catalogue or a high volume of concurrent transactions.
- Passwords are stored as plain text within `library_data.json` rather than being hashed or encrypted, which is acceptable for a college project but not for a production system.
- There is no password-reset workflow; a student or teacher who forgets their password must have an administrator create a new account.
- The three-book borrowing limit applies only to students; teachers can currently borrow an unlimited number of books, which may need to be revisited if the system is used at a larger scale.
- Admin lookup of a student or teacher (“Find a student” / “Find a teacher”) requires only the ID and not a password, so anyone with access to the Admin menu can view any student's or teacher's outstanding fines.
- There is no search or filter facility (by title, author, or genre) — the catalogue is always displayed in full.
- Being a command-line application, it does not provide a graphical user interface, which may be less intuitive for non-technical users.
- The system does not integrate with barcode or RFID scanners for automatic book identification.
- No SMS/email notification facility is available to remind students or teachers about upcoming or missed due dates.

- Backup and recovery of library_data.json must currently be performed manually.

FUTURE APPLICATION OF THE PROJECT

The Library Automation System can be enhanced and extended in several ways to increase its functionality and usability:

- Migrating from JSON file storage to a proper relational database (e.g. SQLite or MySQL) to improve scalability and support concurrent access.
- Development of a graphical user interface (GUI, e.g. Tkinter) or a web-based application (e.g. Flask) to make the system more accessible to non-technical users.
- Adding search and filter functionality so books can be found by title, author, or genre instead of scanning the full list.
- Hashing and securing stored passwords, and adding a proper password-reset workflow.
- Requiring a password (not just an ID) for the Admin “Find a student” / “Find a teacher” lookups, to protect fine and loan details.
- Introducing a configurable borrowing limit for teachers, or per-role limits set by the administrator.
- Automated SMS/email reminders for upcoming and missed due dates.
- Integration of barcode/RFID scanning for faster and more accurate book issue and return.
- Multi-branch support with centralised/networked database access for larger library systems.
- Generation of analytical reports and dashboards on book usage and borrowing trends using data visualisation.
- A native C++ port of the current logic, for comparison with the Python implementation.

BIBLIOGRAPHY

The following resources were referred to during the design, development, and documentation of this project:

1. Python Software Foundation — Python 3 Documentation,
<https://docs.python.org/3/>
2. Python Software Foundation — json — JSON encoder and decoder,
<https://docs.python.org/3/library/json.html>
3. Python Software Foundation — re — Regular expression operations,
<https://docs.python.org/3/library/re.html>
4. Python Software Foundation — datetime — Basic date and time types,
<https://docs.python.org/3/library/datetime.html>
5. Balagurusamy, E. — Programming in Python, McGraw Hill Education.
6. Deitel, P., & Deitel, H. — Python for Programmers, Pearson Education.
7. GeeksforGeeks — Python File Handling, Regular Expressions, and JSON Tutorials, <https://www.geeksforgeeks.org/>
8. W3Schools — Python Tutorial, <https://www.w3schools.com/python/>
9. Graphviz — Graph Visualization Software Documentation,
<https://graphviz.org/documentation/>